Module 7 Final

Filip Arghir

Southern New Hampshire University

CS 330

8/13/2025

The creation of my 3D OpenGL scene focuses on my personal life, artistic design, programming, and user interactivity. My scene features two vehicles, one of which is meant to be mine, and the other is a close friend's. There's also a tree and a lamp post. This scene reflects when I first purchased my car after having my first job, this was the lot of the job which I was working at. I saved up enough money and convinced my parents to pay for half of my car. The car is the Ford Crown Victoria, and I have had the car for quite a while now and adore it. This car, my hard-earned money towards it, and my very close friend who has been there for me for all sorts of troubles are all meant to be the center of this scene.

When designing the scene, I chose objects that could help create a realistic environment in which this scene occurred. The two vehicles use multiple objects to create their realistic composition; multiple complex and primitive objects were used to create the smooth lines needed for the Crown Victoria and Infiniti G35. The vehicles are meant to be the focal point of the scene and put the scene into proper perspective. The tree was also created from a cone and applies a couple of leaves for texturing, and has an elliptical torus meant to be its concrete base. The tree in the photo was meant to simply add contrast to the parking lot, and the same occurs with the same placement. The lamp post in the scene was also used to mimic the placement of the picture and is meant to help incorporate a secondary light source. Its placement next to the vehicles is meant to be that of the parking lot, providing light at nighttime. The picture was originally created in sunlight, and with hopes, I wanted to attempt moonlight and create a lamp that would provide ambient lighting, but this proved to be very difficult. Sunlight ultimately became the ultimate choice.

Movement around the scene is also more than possible; the user can navigate the camera forward, backward, left, right, and use the mouse movement to adjust the camera, and use the

scroll wheel for added speed. This is meant to be that of a first-person experience, but in this case, it feels more like a camera taking a picture of the scene.

The navigation is possible due to tracking the camera position vector as well as the direction vector, and is updated based on user input. The usage of the delta time variable also helps ensure smooth motion and allows users to wander through the scene smoothly. Mouse movement events were created via calculation of yaw and pitch angles, and also updated the camera's front vector using mathematical functions. The scroll wheel, as previously mentioned, allows the zoom level speeds to be adjusted as well.

The design of the scene is meant for the duality of fine smooth movement, for a cinematic feeling, as well as broader movement meant for immersion. The camera and tracking of user inputs, as well as real-time updates to position and direction, all help make it possible to navigate the scene. Users can switch perspectives by using the O and P keys. This is meant to mimic that of an orthographic projection of the scene and a perspective viewing of the scene. The front-facing or the orthographic projection is a front-facing flat representation of the scene with no perspective. This is ideal for truly seeing the scene dead on. The perspective projection, which can be changed via pressing P, allows a real-world depth perception and helps give a realistic feeling to the scene.

Several custom functions were used to handle scene logic; most of these were created for organization as well as to make it a more modular system. I created multiple custom functions to handle the rendering of the objects for this scene. I broke them down per object to make it much simpler. There's a RenderCrownVictoria function that is meant to handle all of the objects needed for the vehicle. When initially creating this scene, I placed all of these functions and objects in the same render class, which made it very hard to debug and keep track of what was

going on. Now the system is easier to follow and understand as well as work on. The scene has essentially been broken down into sections that are much more easily manageable. While the functions aren't entirely reusable, they follow the same formatting, and in turn, the logic is reused, which means that the functions, in a sense, are being reused.